

Lista de projetos para treinar conceitos básicos de programação e Programação Orientada a Objetos.

1. Sistema de Cadastro de Colaboradores com CSV

Descrição

- Crie um sistema orientado a objetos para gerenciar cadastros de colaboradores de uma empresa.
- Sua aplicação deve conter classes básicas (por exemplo, `Colaborador`, `Departamento`, `Endereco`, etc.).
- Devem existir métodos para **inserir**, **editar**, **excluir** e **listar** colaboradores.

Lógica e detalhes

1. **Classe `Colaborador`:**
 - Atributos como `nome`, `email`, `cpf`, `cargo`, `salario`, etc.
 - Use métodos de acesso (getters/setters ou equivalentes) para manipular os atributos.
 - Implemente validações simples (por exemplo, não permitir `nome` vazio, `salario` negativo etc.).
2. **Classe `Departamento`** (opcional, mas interessante):
 - Relacione cada colaborador a um departamento.
 - Pode ter métodos para calcular a soma dos salários de todos os colaboradores do departamento, por exemplo.
3. **Armazenamento em CSV:**
 - Defina um método (ou classe separada) responsável por **salvar** e **ler** dados de um arquivo CSV.
 - Formato CSV (exemplo): cada linha contém os dados de um colaborador.
 - Quando o programa é iniciado, carregue os dados do CSV; quando finaliza ou há alterações relevantes, salve no CSV.
4. **Parâmetros nomeados** (quando disponíveis):
 - Por exemplo, ao criar um novo colaborador, você pode chamar:

```
novoColaborador = new Colaborador(  
    nome="Maria Silva",  
    email="maria.silva@example.com",  
    salario=3500.00  
    // demais parâmetros...  
)
```

Tarefas sugeridas

1. **Cadastrar novo colaborador:** solicite dados e inclua no sistema, persistindo em CSV.
2. **Listar colaboradores:** exibir em tela ou console todos os colaboradores, com opção de filtrar por departamento.
3. **Editar colaborador:** dado um identificador (por exemplo, CPF), altere seus dados.
4. **Excluir colaborador:** remover do arquivo CSV.
5. **Relatórios:** total de salários por departamento, quantidade de funcionários por departamento, etc.

2. Aplicativo de Biblioteca com JSON

Descrição

- Crie um programa que simule o gerenciamento de uma biblioteca.
- O sistema deve armazenar **livros** e **usuários** em arquivos JSON.
- Exercite herança e polimorfismo, caso queira diferenciar subclasses de livros (por exemplo, livros físicos e eBooks).

Lógica e detalhes

1. **Classes principais:**
 - **Livro**: com atributos como `titulo`, `autor`, `isbn`, etc.
 - **Usuario**: com atributos como `nome`, `email`, `listaEmprestimos`, etc.
2. **Exemplo de herança (opcional):**
 - Subclasse **LivroFisico** (pode ter atributo adicional `exemplarEmEstoque`) e
 - Subclasse **LivroDigital** (pode ter atributo `urlDownload` ou `tamanhoArquivo`).
3. **Arquivo JSON:**
 - Estruture os dados em JSON para fácil escrita/leitura. Por exemplo, "`livros.json`" e "`usuarios.json`".
 - Use métodos ou classes utilitárias para carregar e salvar o JSON.
4. **Funcionalidades:**
 - Realizar empréstimo de um livro para um usuário (validando se há exemplares disponíveis, datas de devolução, etc.).
 - Calcular multas por atraso, caso queira adicionar lógica adicional.
 - Gerar relatório de livros atualmente emprestados.

Tarefas sugeridas

1. **Cadastro de livros** (salvo em JSON).
 2. **Cadastro de usuários** (também salvo em JSON).
 3. **Empréstimo e devolução**: mantenha um registro atualizado no JSON de quem pegou qual livro.
 4. **Relatório**: listar quantos livros ainda estão disponíveis, quem está com mais livros emprestados, etc.
-

3. Sistema de Reservas de Quartos (Hotel)

Descrição

- Faça um sistema que gerencie **quartos de hotel**, permitindo reservar, fazer check-in e check-out.
- Utilize composição de objetos (por exemplo, um **Hotel** que tem vários **Quarto**, e cada **Quarto** pode ter estado "reservado", "ocupado", "disponível").

Lógica e detalhes

1. **Classes:**
 - **Hotel** (contém uma lista ou array de **Quarto**).
 - **Quarto** (tem atributos como `numero`, `capacidade`, `diaria`, `disponivel`, etc.).
 - **Reserva** (poderia ter data de entrada, data de saída, o usuário que reservou).
2. **Métodos:**
 - `reservar(quarto, usuario, dataInicio, dataFim)`.

- `checkin(quarto, usuario, dataEntrada)`.
- `checkout(quarto, dataSaida)` – retorna valor total a pagar, por exemplo.

3. Armazenamento:

- Pode armazenar em CSV ou JSON, conforme preferir.
- Se quiser algo mais complexo, salve **Reservas** em arquivo separado.

Tarefas sugeridas

1. **Controle de datas:** verificar se o quarto está disponível no período solicitado.
 2. **Cálculo do valor** da reserva, com base na quantidade de diárias.
 3. **Relatório** de ocupação atual (quais quartos estão ocupados e quais estão livres).
 4. **Parâmetros nomeados** (quando a linguagem suportar) na hora de criar objetos ou chamar métodos.
-

4. Jogo de Cartas com Foco em POO e Lógica

Descrição

- Desenvolva um jogo simples de cartas (ex.: Blackjack, Buraco simplificado ou qualquer outro) usando várias classes para representar **Baralho**, **Carta**, **Jogador**, etc.
- A ênfase é em **lógica de jogo** e **modelagem** das interações via POO.

Lógica e detalhes

1. **Classe Carta:** atributos como **naipe** (copas, espadas, etc.) e **valor** (às, números, etc.).
2. **Classe Baralho:** gerencia um conjunto de cartas, com métodos de **embaralhar**, **distribuir**, etc.
3. **Classe Jogador:** mantém uma lista de cartas na mão, com métodos para **pegarCarta**, **jogarCarta**, etc.
4. **Fluxo do jogo:** implemente a lógica do jogo escolhido (exemplo: somar valores até 21 no Blackjack, ou formar sequências no Buraco).
5. **Salvamento** (opcional): salve o andamento do jogo em JSON para poder retomar depois, por exemplo.

Tarefas sugeridas

1. **Inicializar jogo** e distribuir cartas aos jogadores.
 2. **Executar rodadas**, imprimindo a cada rodada as cartas de cada jogador (ou ao menos quantas cartas restam).
 3. **Regras de término:** definir quem ganhou, e exibir o resultado final.
-

5. Plataforma de Gestão Financeira (CRUD + Relatórios)

Descrição

- Crie um sistema de **gestão de despesas e receitas** pessoais.
- Deve haver classes como **Transacao** (com atributos de valor, data, descrição, tipo), **ContaBancaria**, etc.

Lógica e detalhes

1. Classe **Transacao**:

- Atributos: **valor**, **data**, **tipo** (**despesa** ou **receita**), **categoria** (alimentação, transporte etc.).
- Métodos de validação (não permitir valor negativo para receita, ou algo similar).

2. Classe **ContaBancaria**:

- Mantém uma lista interna de transações.
- Métodos para adicionar (**adicionarTransacao(...)**) e remover (**estornarTransacao(...)**).
- Método para calcular saldo atual, total de despesas, total de receitas, etc.

3. **Armazenamento**:

- CSV ou JSON, gravando as transações ou o estado da conta.

4. **Relatórios**:

- Relatório de total de despesas por categoria (por exemplo, quanto foi gasto em alimentação).
 - Listagem das 5 maiores despesas do mês.
 - Opção de exportar dados para outro CSV ou JSON.
-

6. Sistema de Inventário para Loja Online

Descrição

- Desenvolva um sistema que gerencie o **estoque** de produtos de uma loja virtual.
- Cada **produto** terá atributos como: nome, SKU ou código identificador, quantidade em estoque, preço de custo, preço de venda, etc.
- A aplicação deve possibilitar a leitura e a gravação dos dados de estoque em **CSV** ou **JSON**.

Lógica e detalhes

1. **Classes**:

- **Produto**: com atributos e métodos para manipular sua descrição, estoque, reajustes de preço.
- **Estoque** ou **Inventario**: responsável por armazenar (em uma lista/array) os produtos, e por métodos de inclusão, remoção e pesquisa.

2. **Funcionalidades**:

- **Cadastrar produto**: permitir inserir um novo produto, checando se o código SKU já existe.
- **Atualizar estoque**: baixar ou acrescentar unidades vendidas/recebidas.
- **Relatório de estoque baixo**: exibir produtos com estoque abaixo de um determinado limite.

3. **Persistência**:

- Carregar os produtos de um arquivo CSV/JSON ao iniciar.
- Salvar automaticamente após cada modificação importante (por exemplo, após cadastrar ou atualizar estoque).

4. **Possíveis extensões**:

- Cálculo de **valor total** em estoque.
 - Registro de histórico de compras ou vendas (com datas, quantidades, etc.).
 - **Parâmetros nomeados** ao criar instâncias de **Produto**.
-

7. Gestor de Playlist Musical

Descrição

- Construa um programa POO que gerencie **playlists** e **músicas**.
- Cada playlist pode conter várias músicas; cada música tem atributos como título, artista, duração, etc.
- Os dados devem ser armazenados em **JSON** (uma ou várias playlists em um único arquivo, por exemplo).

Lógica e detalhes

1. **Classes:**
 - **Musica:** atributos (título, artista, duração, gênero musical).
 - **Playlist:** lista de músicas. Pode ter métodos para adicionar, remover, ordenar músicas, etc.
 2. **Funcionalidades:**
 - **Criar playlist:** dar nome à playlist (ex.: “Rock Anos 80”) e começar vazia.
 - **Adicionar música** a uma playlist específica (validando duplicados se desejar).
 - **Buscar** músicas (por nome, artista, etc.).
 3. **JSON:**
 - Um arquivo **playlists.json** que armazene todas as playlists existentes ou um arquivo por playlist, a critério do projeto.
 - Após adicionar/remover música, atualizar o arquivo.
 4. **Relatórios / Extensões:**
 - Exibir **duração total** da playlist.
 - Listar playlists com mais de “N” músicas.
 - Permitir reordenar as músicas de acordo com critério (nome, artista, duração).
-

8. Sistema de Votação / Pesquisa de Opinião

Descrição

- Monte um **sistema de pesquisa** em que usuários possam votar em diversas perguntas (enquetes).
- Cada enquete tem várias opções de resposta. O sistema registra cada voto e, ao final, mostra estatísticas.

Lógica e detalhes

1. **Classes:**
 - **Enquete:** contém a pergunta e as possíveis respostas (ex.: “Qual linguagem de programação você prefere?”, com X opções).
 - **Voto:** registra quem votou (ou ID anônimo) e em qual opção.
 - **Opcional:** **Usuario** para representar uma pessoa que pode votar (caso precise de login ou algo mais complexo).
2. **Fluxo:**
 - Criar enquetes e opções.
 - Receber votos (pode exigir que cada usuário só vote uma vez).
 - Calcular percentuais de cada opção.
3. **Armazenamento:**
 - Pode ter um arquivo CSV ou JSON para armazenar:

- As perguntas e opções.
- A contagem de votos ou lista de quem votou.

4. Recursos de lógica:

- Caso decida não armazenar cada voto individualmente, pode gravar apenas contadores por opção.
- Gerar um **relatório final** mostrando a opção mais votada, etc.

5. Estendendo:

- Permitir **múltiplas enquetes** simultâneas.
 - Adicionar data de início/fim de uma enquete.
 - **Parâmetros nomeados** para criar as opções de forma clara.
-

9. Sistema de Matrículas em Cursos

Descrição

- Crie um **sistema educacional** para gerenciar cursos, alunos e matrículas.
- Cada curso tem um nome, código, carga horária, e uma lista de alunos matriculados.

Lógica e detalhes

1. Classes:

- **Aluno**: atributos como nome, matrícula (ID), email, etc.
- **Curso**: atributos como nome do curso, código único, carga horária, professor responsável, etc.
- **Matricula**: relaciona **Aluno** e **Curso**, podendo ter também uma data de início.

2. Funcionalidades:

- **Cadastrar aluno** em um curso, verificando se ainda há vagas (se quiser impor limite de vagas).
- **Listar alunos** de um curso, ou cursos de um aluno.
- Caso exista nota ou avaliações, poderia gerenciar também as médias dos alunos.

3. Persistência:

- Utilize CSV ou JSON para manter:
 - A lista de alunos cadastrados.
 - A lista de cursos.
 - Quem está matriculado em qual curso.

4. Relatórios / Extensões:

- Calcular quantidade de alunos em cada curso.
- Exibir alunos reprovados/aprovados (se houver notas).
- Permitir desvincular um aluno do curso.

5. Parâmetros nomeados:

- Ao criar ou atualizar **Aluno**, ou ao instanciar um **Curso**.
-

10. Simulador de Fila de Processos (ou Fila de Atendimento)

Descrição

- Modele uma **fila** em que cada item seja um “processo” ou “chamado” que precisa ser atendido.
- Cada processo tem tempo de chegada, tempo de execução, prioridade (ou outro critério que queira simular).

- Pode ser aplicado a um cenário de fila de **atendimento bancário**, fila de **processamento de tasks** etc.

Lógica e detalhes

1. Classes:

- **Processo** ou **Tarefa**: atributos como ID, descrição, prioridade, tempo necessário para ser concluído.
- **FilaProcessos** ou **Agendador**: mantém uma lista de processos, ordenados por algum critério (FIFO, prioridade etc.).

2. Fluxo:

- Inserir processos na fila (poderia ser manual ou carregado de arquivo).
- “Executar” passo a passo (por exemplo, reduzir o tempo de execução de cada processo).
- Marcar o processo como finalizado quando o tempo de execução chegar a zero.

3. Armazenamento:

- CSV/JSON para listar processos pendentes, finalizados, etc.
- Ao iniciar, ler o estado atual da fila (se desejar simular persistência contínua).

4. Desafios lógicos:

- Implementar diferentes **políticas de escalonamento** (ex.: Round Robin, Prioridade, FIFO).
- Criar **estatísticas** ao final, por exemplo, tempo médio na fila, quantos processos concluídos, etc.

5. Extensões:

- Permitir ao usuário escolher o método de escalonamento no início.
- Salvar logs de execução para posterior análise.

Observações finais

- Em **todas** as propostas acima, você exercitará **construção de classes**, **uso de objetos**, **encapsulamento** e (quando quiser) **herança**.
- Para cada projeto, **crie métodos** que exijam **lógica** de negócio: verificação de disponibilidade, cálculos de valores, persistência em arquivos etc.
- **Parâmetros nomeados** podem ser usados na criação de instâncias ou na chamada de métodos (linguagens como Python, Kotlin, Swift suportam isso).
- Se a linguagem escolhida não tiver suporte direto a parâmetros nomeados, ainda assim organize bem os construtores e métodos para manter o código claro.

Esses projetos são apenas **enunciados gerais**, mas cada um pode ser expandido significativamente:

- Adicione **tratamento de exceções** (caso a linguagem suporte).
- Inclua **testes unitários** dos métodos mais importantes.
- Utilize **padrões de projeto** (Strategy, Observer, Singleton) se quiser aprofundar.

Com estes projetos, você cobrirá:

- Conceitos de **POO** (classes, objetos, encapsulamento, composição, herança, polimorfismo).
- **Persistência** básica em arquivos CSV ou JSON.
- **Lógica de programação** para validações, cálculos e restrições de cada domínio.

Projeto DESAFIO: Sistema de Logística e Roteirização de Entregas

Imagine que você precisa construir um **simulador** de uma empresa de entregas, que precisa gerenciar *pedidos, veículos, motoristas (entregadores)* e *rotas* para realizar as entregas da forma mais otimizada possível (ou, ao menos, com alguma lógica interessante).

Visão Geral

- O projeto consiste em **planejar entregas** a vários endereços, cada um com um pedido (pacote) a ser levado.
- Você terá de **modelar** dados como:
 - **Cidade ou mapa** com pontos (endereços) e as distâncias entre eles.
 - **Lista de pedidos** (cada pedido tem um endereço de destino, peso, tamanho, prazo máximo, etc.).
 - **Frota de veículos** (capacidade, custo de combustível, velocidade média, etc.).
 - **Entregadores** (motoristas) e suas restrições de jornada (ex. máximo de horas/dia).
- O sistema deve **ler/gravar** seus dados em **arquivos CSV** ou **JSON** para que essas informações sejam persistidas e recuperadas a qualquer momento.

Com isso, você enfrentará **desafios** como:

1. **POO avançada**: Várias classes relacionadas (Pedido, Veiculo, Rota, Entregador, Mapa, etc.).
2. **Estrutura de dados e algoritmos**: Precisar armazenar e manipular grafos (pontos e distâncias) e possivelmente implementar busca de caminhos (Dijkstra, BFS/DFS ou até heurísticas de caixeiro-viajante, se quiser ir além).
3. **Validação de dados**: Certificar-se de que as distâncias, prazos e capacidades estão dentro de limites coerentes.
4. **Relatórios e estatísticas**: Exibir como foram distribuídas as entregas, qual o custo total, tempo total, etc.

A seguir, detalho sugestões de classes e funcionalidades para deixar o projeto robusto.

Classes Principais e Atributos

1. **Localizacao** (ou **PontoMapa**)
 - Representa um ponto no mapa, podendo ter atributos como **nomeDoLugar** ou **endereço**, além de **coordenadas** (x, y) se você modelar graficamente, ou um **ID** e **lista de distâncias** para outros pontos.
 - Métodos:
 - **getDistancia(outraLocalizacao)**: retorna a distância entre os dois pontos (com base na matriz de distâncias ou algo similar).
2. **Mapa**
 - Contém uma **lista** (ou dicionário) de **Localizacao**.
 - Armazena distâncias entre esses locais (pode ser uma matriz NxN, caso tenha N pontos).
 - Métodos:
 - **adicionarLocalizacao(...)**

- `definirDistancia(localA, localB, distancia)`
- `encontrarMenorCaminho(origem, destino)`: aqui você pode implementar **Dijkstra** ou um algoritmo mais simples caso seja um grafo pequeno.

3. Pedido

- Atributos:
 - `idPedido`
 - `localDeEntrega` (objeto `Localizacao`)
 - `prazoMaximo` (ex.: data/hora ou número de horas que se pode levar)
 - `peso` ou `volume` (para checar se cabe em determinado veículo)
 - `status` (ex.: “pendente”, “em entrega”, “entregue”)
- Métodos:
 - `marcarComoEntregue()`
 - Validações (peso negativo, prazos inconsistentes, etc.).

4. Veiculo

- Atributos:
 - `idVeiculo`
 - `capacidadePeso`
 - `velocidadeMedia` (km/h, por exemplo)
 - `custoPorKm` ou `consumoCombustivel`
 - `status` (disponível, em rota, manutenção, etc.)
- Métodos:
 - `podeTransportar(pesoSomado)` (retorna se aguenta a carga total).
 - `calcularTempoDePercurso(distancia)`: a partir da velocidade média, calcula quanto tempo levará para percorrer uma distância.
 - `calcularCustoDePercurso(distancia)`: ex.: `distancia * custoPorKm`.

5. Entregador (ou Motorista)

- Atributos:
 - `nome`
 - `identificador`
 - `limiteHorasDiarias`
 - `horasTrabalhadasNoDia`
 - `veiculoAtual` (pode referenciar um `Veiculo` em uso)
- Métodos:
 - `podeDirigirPor(horas)`: verifica se `horasTrabalhadasNoDia + horas <= limiteHorasDiarias`.
 - `atribuirVeiculo(veiculo)`.

6. Rota

- Representa o **planejamento** de entrega:
 - Quais pedidos serão entregues
 - Em qual **ordem** (sequência de pontos)
 - Qual entregador/veículo fará a rota
- Atributos:
 - `listaDePedidos`
 - `distanciaTotal`
 - `tempoPrevisto`
 - `custoEstimado`
 - `entregadorResponsavel`
- Métodos:

- `adicionarPedido(pedido)`
 - `calcularDistanciaTotal()`: percorre a sequência de endereços e soma as distâncias, usando o objeto `Mapa`.
 - `calcularTempoECusto()`: combina métodos do `Veiculo`, velocidade, distâncias, etc.
7. **SistemaLogistica** (ou algo similar)
- Classe principal que **orquestra** tudo:
 - Contém `Mapa` (com todos os pontos e distâncias).
 - Listas de `Pedido`, `Veiculo`, `Entregador`, `Rota`.
 - Métodos (exemplos):
 - `carregarDadosDeJSON(...)`, `salvarDadosEmJSON(...)`
 - `gerarRotaOtimizada(List<Pedido>, Veiculo)`: aqui entra a lógica de roteirização (por exemplo, caminho mais curto que passe em todos os endereços, ou heurística se preferir).
 - `alocarEntregas()`: dado um conjunto de pedidos, a frota de veículos e motoristas disponíveis, tente **distribuir** os pedidos em várias rotas, respeitando capacidade, tempo, etc.
-

Funcionalidades / Desafios

1. **Persistência** (CSV ou JSON)
 - Armazenar:
 - **Mapa** (lista de locais e distâncias)
 - **Lista de Pedidos**
 - **Lista de Veículos e Entregadores**
 - Implementar métodos robustos de **leitura** e **escrita** nos arquivos.
 - Lidar com formatações, validações, e possíveis corrupções nos arquivos (desafio extra).
2. **Planejamento de Rotas**
 - **Desafio-chave**: como decidir a ordem de entrega para minimizar distância/tempo/custo?
 - Pode usar um algoritmo como:
 - **Dijkstra** para achar o menor caminho entre dois pontos (quando estiver indo de um endereço a outro).
 - **Heurística** do “caminho mais próximo” para ir percorrendo vários pedidos (simples, mas interessante).
 - Versão simplificada do **Problema do Caixeiro Viajante** (TSP), se quiser algo mais avançado.
 - Mesmo que não seja a solução “ótima” do TSP, implementar qualquer heurística já é um grande desafio e envolve lógica mais complexa.
3. **Restrição de Capacidade do Veículo**
 - Antes de montar uma rota, some o peso/volume de todos os pedidos que pretende colocar no veículo e verifique se não excede `capacidadePeso`.
 - Caso exceda, divida em duas rotas ou use outro veículo.
4. **Restrição de Horas do Entregador**
 - Calcular o tempo previsto da rota.
 - Checar se o entregador ainda **pode dirigir** esse tanto de horas no dia (se quiser simular algo realista).
 - Se não puder, ou troca de motorista ou faz parte da rota em outro dia.
5. **Relatórios**
 - **Relatório de rotas geradas**: para cada rota, qual a ordem de entrega, distância total, custo estimado, etc.

- **Relatório de pedidos entregues:** status, horário de entrega, motoristas responsáveis.
- **Tempo médio** de entrega, ou **percentual** de entregas atrasadas, etc.

6. Interface (console)

- Menus para:
 - Cadastrar/editar **Localizacao**
 - Definir distâncias entre localidades
 - Inserir novos pedidos
 - Inserir novos veículos, entregadores
 - **Gerar rotas** automaticamente com base nos pedidos pendentes
 - Exibir relatórios
-

Passos para Execução / Montagem

1. Modelagem das Classes:

- Crie cada classe em seu próprio arquivo, ou em módulos, dependendo da linguagem.
- Planeje bem os relacionamentos: **Pedido** -> **Localizacao**, **Entregador** -> **Veiculo**, **Rota** -> **Pedido**, etc.

2. Implementação Inicial:

- Talvez começar pelo **Mapa** (para poder ter distâncias) e **Localizacao**.
- Depois **Pedido**, **Veiculo**, **Entregador**.
- Deixe a parte de **geração de rotas** para um momento posterior.

3. Algoritmos de Caminho:

- Use Dijkstra ou BFS se as distâncias forem unitárias ou se quiser algo mais simples.
- Se preferir, mantenha tudo em uma matriz NxN de distâncias.
- Para a roteirização (vários endereços), faça uma **função** ou **método** de heurística que gera a ordem de visita.

4. Persistência:

- Configure métodos de **carregarArquivo()** e **salvarArquivo()**.
- Defina o formato CSV ou JSON.

Exemplo JSON:

```
{
  "localizacoes": [
    { "id": 1, "nome": "Centro", "distancias": { "2": 10.5, "3": 2.1 } },
    ...
  ],
  "pedidos": [
    { "idPedido": 101, "localDeEntrega": 3, "peso": 5.2, "status": "pendente" },
    ...
  ]
}
```

■

- Ler e escrever deve ser feito cuidadosamente (usar alguma biblioteca ou escrever manualmente).

5. Teste e Validação:

- Teste rotas com poucos pedidos e verifique se o tempo e distância batem.
- Adicione casos de teste para exceder a capacidade do veículo, exceder horas de trabalho, etc.

Por que é Desafiador?

1. **Estrutura Multiclasse:** são muitas entidades interdependentes (Mapa, Pedido, Veículo, Entregador, Rota...).
2. **Algoritmos:** a roteirização pode ser bem complexa — resolver ou aproximar problemas de caminho mínimo ou TSP gera muito desafio.
3. **Validações e Restrições** (capacidade do veículo, horas do motorista, prazos dos pedidos).
4. **Persistência:** leitura e escrita de vários tipos de dados (loais, distâncias, pedidos, veículos) num formato coerente.
5. **Relatórios:** diversas estatísticas e visualização final de como as entregas foram realizadas.

Esse projeto permite evoluir bastante:

- Você pode começar simples (rota de um veículo só, sem restrição de horas, sem TSP) e ir adicionando complexidade.
- Pode incluir lógica de **agenda** (data/hora real) para cada entrega.
- Pode criar um **simulador de tempo** em que cada rota vai “consumindo” horas e atualizando o status de pedidos.